CSA0307 — DATA STRUCTURES

RS Vigneshwaran | Register Number: 192521307

C PROGRAMMING — ANSWERS

Course Code: CSA0307 — Data Structures

Name: RS Vigneshwaran

Register Number: 192521307

Question Set

1. What is a pointer in C? Explain its relevance. (5 Marks)
2. Write a C program to check whether a number is a palindrome. (10 Marks)
3. Discuss the use of recursion in the Fibonacci series. (5 Marks)

1. What is a pointer in C? Explain its relevance. (5 Marks)

Definition

A pointer in C is a variable whose value is the memory address of another object or function. An ordinary variable stores a data value directly, whereas a pointer stores the location at which a value can be found. Pointers are fundamental to C because the language gives programmers direct control over memory addresses and supports efficient manipulation of arrays, strings, dynamically allocated memory, and linked data structures.

Declaration and initialization

```
int number = 25;
int *ptr = &number;
```

In this example, number is an integer variable and ptr is a pointer to an integer. The ampersand operator (&), called the address-of operator, obtains the address of number. The asterisk in the declaration int *ptr indicates that ptr points to an int. A pointer should normally be initialized before it is used.

The dereference operator

```
printf("%d\\n", number); /* 25 */
printf("%p\\n", (void *)ptr); /* address of number */
printf("%d\\n", *ptr); /* 25 */
*ptr = 40; /* changes number */
```

The unary * operator, when applied to a valid pointer, dereferences it: it accesses the object stored at the pointed-to address. Therefore, assigning 40 to *ptr changes number itself. The %p conversion specifier is used to print an address, and the pointer is converted to void * for a portable printf call.

Pointers: memory model and operators

Conceptual memory representation

Suppose number contains 25 and is stored at a hypothetical memory address 1000. The pointer ptr contains the address 1000, not the integer value 25 itself. Dereferencing ptr accesses the integer at address 1000. Actual addresses are determined by the execution environment and should never be assumed to be fixed.

Expression Meaning
number Value stored in the integer variable
&number Address of the integer variable
ptr Address stored in the pointer
*ptr Value of the object at the stored address

Important pointer operators

● Address-of (&): obtains the address of an object, for example &number.

● Dereference (*): accesses or modifies the object referenced by a valid pointer, for example *ptr. ● Pointer arithmetic: allows movement between elements of an array. Incrementing an int pointer advances it by sizeof(int) bytes, not necessarily by one byte.

Pointers and arrays

In most expressions, an array name is converted to a pointer to its first element. If int values[3] = {10, 20, 30}; then values points to values[0] in an expression, and *(values + 1) accesses values[1]. Pointer arithmetic is only defined within the same array object or one position beyond its end; the one-past pointer must not be dereferenced.

```
int values[3] = {10, 20, 30};
int *p = values;
printf("%d\\n", *p); /* 10 */
printf("%d\\n", *(p + 1)); /* 20 */
```

Relevance and applications of pointers

1. Passing addresses to functions

C passes function arguments by value. A pointer can be passed when a function needs to modify an object belonging

to its caller. The function receives a copy of the address, but dereferencing that address lets it update the original object.

```
void set_to_zero(int *value)
{
    if (value != NULL)
        *value = 0;
}


int score = 85;
set_to_zero(&score); /* score becomes 0 */
```

2. Dynamic memory allocation

Pointers store the addresses returned by dynamic-memory functions such as malloc(), calloc(), and realloc(). This allows a program to request memory while it is running, when the required size may not be known in advance. Memory obtained dynamically must be released with free() when no longer needed.

```
int *data = malloc(5 * sizeof *data);
if (data != NULL) {
    /* use data[0] through data[4] */
    free(data);
    data = NULL;
}
```

The example requires #include <stdlib.h>. Allocation failure must be checked. Setting a pointer to NULL after freeing can help avoid accidentally reusing that same pointer, although other aliases to the freed memory would still be invalid.

3. Data structures and systems programming

Pointers connect nodes in linked lists, trees, and graphs; represent dynamic relationships; and support APIs that operate on buffers. They are also used in operating systems, embedded systems, device drivers, and other low-level software where memory layout matters.

Pointer safety, advantages, and conclusion

Common pointer errors

● Uninitialized pointer: contains an indeterminate value and must not be dereferenced. ● Null pointer dereference: NULL represents a pointer that intentionally refers to no object; it must be checked before access. ● Dangling pointer: refers to an object whose lifetime has ended, such as memory that has already been freed. ● Out-of-bounds access: pointer arithmetic or dereferencing outside the valid array object causes undefined behaviour.

● Memory leak: dynamically allocated memory is no longer reachable and cannot be freed.

Good practices

Initialize pointers, validate allocation results, match pointer types to the objects being accessed, respect object lifetimes, and release dynamically allocated memory exactly as required by the ownership design. Use const when a function should not modify the pointed-to object. Never assume that an address has a particular numeric value.

Advantages

Pointers enable efficient data sharing without copying large objects, support in-place modification, make dynamic memory management possible, and allow flexible data structures. These capabilities are a major reason C is widely used in systems and embedded programming.

Conclusion

A pointer is a variable that stores a memory address. The address-of operator obtains an address, while dereferencing accesses the object at that address. Pointers are relevant to function interfaces, arrays, strings, dynamic memory, and linked data structures. Their power must be balanced with careful initialization, bounds checking, and lifetime management.

2. Write a C program to check whether a number is a palindrome. (10 Marks)

Aim and definition

Aim: To write a C program that checks whether a non-negative integer is a palindrome. A palindrome number reads the same from left to right and right to left. Examples include 0, 7, 121, 1331, and 4554. The number 123 is not a palindrome because its reverse is 321.

Logic

The program extracts the last digit of the input using the remainder operator (%). It appends that digit to a reverse accumulator by multiplying the accumulator by 10 and adding the digit. Integer division by 10 removes the last digit from the working number. The process continues until no digits remain.

Algorithm

● Start.
● Read a non-negative integer and validate the input.
● Save the input in original so it remains available for comparison.
● Initialize reverse to zero.
● While number is greater than zero, calculate digit = number % 10.
● Update reverse = reverse * 10 + digit.

- Remove the last digit using number = number / 10.
- Compare reverse with original and print the result.
- Stop.

Pseudocode

```
READ number
original = number
reverse = 0
WHILE number > 0
    digit = number MOD 10
    reverse = reverse * 10 + digit
    number = number DIV 10
END WHILE
IF reverse == original
    PRINT palindrome
ELSE
    PRINT not palindrome
END IF
```

Palindrome program: complete C implementation

C program

```
#include <stdio.h>

int main(void)
{
    int number, original, digit;
    long long reverse = 0;

    printf("Enter a non-negative integer: ");
    if (scanf("%d", &number) != 1 || number < 0) {
        printf("Invalid input. Please enter a non-negative integer.\\n");
        return 1;
    }

    original = number;

    while (number > 0) {
        digit = number % 10;
        reverse = reverse * 10 + digit;
        number = number / 10;
    }
```

```
    if (reverse == original) {
        printf("%d is a palindrome number.\\n", original);
    } else {
        printf("%d is not a palindrome number.\\n", original);
    }

    return 0;
}
```

Explanation of variables

- number: working copy of the input; its last digit is removed during each loop iteration.
- original: preserves the initial input for the final comparison.
- digit: stores the last digit extracted from number.
- reverse: accumulates the digits in reverse order. It is declared as long long to provide a wider intermediate range than int on common systems.

The program checks whether scanf successfully reads an integer and rejects negative values. This makes the accepted input range explicit. For very large inputs, the reverse accumulator can still overflow; a production-grade implementation would add an overflow check or process the input as a string.

Palindrome: worked examples and testing

Worked example 1: input 121

Iteration	number	digit = number % 10 reverse after update
Initial	121	— 0
1	12	1 1
2	1	2 12
3	0	1 121

When the loop ends, reverse is 121 and original is also 121. Since the two values are equal, the program reports that 121 is a palindrome.

Worked example 2: input 123

Iteration	number	digit = number % 10 reverse after update
Initial	123	— 0
1	12	3 3

2	1	2 32
3	0	1 321

The reverse is 321, which differs from the original value 123. Therefore, the program reports that 123 is not a palindrome.

Sample outputs

```
Enter a non-negative integer: 121
121 is a palindrome number.


Enter a non-negative integer: 123
123 is not a palindrome number.


Enter a non-negative integer: 0
0 is a palindrome number.
```

Why save the original number?

The loop modifies number until it becomes zero. Without saving the original value first, the program would have no reliable way to compare the reversed digits against the original input. The original variable therefore preserves the value needed for the final decision.

Palindrome: correctness, complexity, and edge cases

Correctness reasoning

At each iteration, the algorithm takes the final digit from the remaining number and appends it to reverse. Multiplication by 10 shifts the digits already accumulated one decimal position to the left; adding digit places the newly extracted digit at the units position. Integer division by 10 then discards the digit that has been processed. Consequently, after all iterations, reverse contains the digits of the original input in reverse order.

Complexity analysis

Let d be the number of decimal digits in the input. Each loop iteration processes exactly one digit, and the loop runs d times for a positive integer. Therefore, time complexity is O(d), equivalently O(log n) for positive n. The algorithm uses a fixed number of variables, so auxiliary space complexity is O(1).

Boundary and edge cases

● Input 0: the loop executes zero times, reverse remains 0, and the result is correctly classified as a palindrome. ● Single-digit inputs such as 5: every non-negative single-digit integer is a palindrome. ● A palindrome with even

digits, such as 1221: the reverse equals the original.

- A non-palindrome such as 120: its reverse is 21, so it is not a palindrome.
- Negative input: rejected by this version because the assignment program explicitly accepts non-negative integers.
- Invalid textual input: if scanf cannot read an integer, the program prints an error and exits.

Alternative approach

A string-based solution can compare characters from both ends and is useful when inputs may be extremely large. However, the arithmetic approach above is suitable for demonstrating remainder, integer division, loops, and numeric reversal in a basic C programming assignment.

3. Discuss the use of recursion in the Fibonacci series. (5 Marks)

Definition of Fibonacci series

The Fibonacci series is a sequence in which each term, beginning with the third term, is the sum of the two terms immediately before it. Under the common zero-based convention, the sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, and so on.

Mathematical recurrence

```
F(0) = 0
F(1) = 1
F(n) = F(n - 1) + F(n - 2), for n >= 2
```

The recurrence defines a problem in terms of smaller instances of the same problem. To calculate F(n), the function calculates F(n − 1) and F(n − 2), then adds the returned values.

Meaning of recursion

Recursion is a programming technique in which a function calls itself. Every correct recursive solution needs base case(s), which terminate recursion, and a recursive case, which reduces the problem toward a base case. Without a terminating base case, recursive calls may continue until the program exhausts its call stack.

Recursive C function

```
int fibonacci(int n)
{
    if (n == 0)
        return 0; /* base case */
    if (n == 1)
        return 1; /* base case */
```

```
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

This function assumes n is non-negative. In a complete program, input should be validated before calling the function, and the return type must be large enough for the requested term.

Fibonacci recursion: execution, benefits, and limitations

How the recursive function works

Consider fibonacci(4). It calls fibonacci(3) and fibonacci(2). The call fibonacci(3) calls fibonacci(2) and fibonacci(1); fibonacci(2) calls fibonacci(1) and fibonacci(0). The base cases return 1 and 0, and the results are added as the recursive calls finish.

```
fibonacci(4)
= fibonacci(3) + fibonacci(2)
= [fibonacci(2) + fibonacci(1)]
  + [fibonacci(1) + fibonacci(0)]
= [(1 + 0) + 1] + [1 + 0]
= 3
```

Example sequence

Calling the function for n = 0, 1, 2, 3, 4, 5, 6 produces 0, 1, 1, 2, 3, 5, 8. Note that F(4) equals 3 under the zero-based convention; if a question counts the first term as position 1, it is important to state the convention being used.

Advantages of recursion

● The implementation closely matches the mathematical recurrence.

● The code is concise and helps illustrate base cases, recursive cases, and the call stack. ● It is a useful teaching example for understanding how a problem can be decomposed into smaller subproblems.

Limitations of simple recursion

The naive recursive implementation repeats calculations. For example, fibonacci(2) is evaluated more than once while calculating fibonacci(4), and repeated work increases rapidly for larger n. Its running time is exponential, commonly described as O(2^n), and the deepest chain of calls uses O(n) stack space. Large inputs may therefore be slow or exhaust the call stack.

Fibonacci: improved methods and conclusion

Memoization

Memoization stores results that have already been calculated. When the same Fibonacci term is requested again, the

saved result is returned instead of recursively recalculating it. This reduces the time complexity to O(n), with O(n) additional memory for the cache and call stack.

Iterative alternative

```
int fibonacci_iterative(int n)
{
    if (n <= 1)
        return n;

    int previous = 0;
    int current = 1;

    for (int i = 2; i <= n; i++) {
        int next = previous + current;
        previous = current;
        current = next;
    }
    return current;
}
```

The iterative version calculates each term once and uses O(n) time with O(1) auxiliary space. As with any integer implementation, the chosen type has a maximum representable value, so sufficiently large Fibonacci terms can overflow. A production implementation should validate n and choose a suitable numeric representation.

Comparison

Method	Time complexity	Extra space Main characteristic
Naive recursion	Exponential, about O(2^n)	O(n) stack Simple and closely follows recurrence
Memoized recursion	O(n)	O(n) Avoids repeated calculations O(1) Efficient for sequential calculation
Iteration	O(n)	

Conclusion

Recursion is a natural way to implement the Fibonacci series because each term depends on the two preceding terms. The base cases F(0) = 0 and F(1) = 1 stop the recursive calls, while the recursive case adds F(n − 1) and F(n − 2). The simple recursive function is clear and useful for learning, but memoization or iteration is preferable for efficient computation.